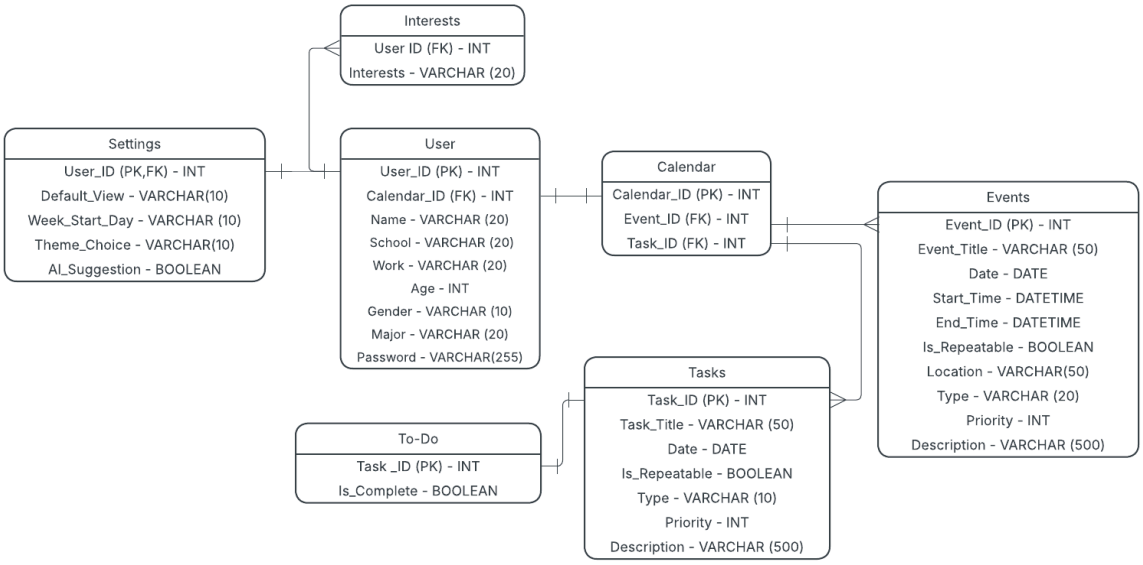
**README**
**WEB CALENDAR APP FOR STUDENTS**
**Section 01 Group 4**

**App Summary**:

College students have trouble keeping track of everything that is going on socially, academically, and professionally. Our personas included those of students at BYU who have jobs, are in rigorous majors, have active social lives, and have church callings. After talking with several individuals that matched our personas we decided to make a calendar app that consolidates the tasks and events for BYU specific students. Most importantly linking calendars from several school, work, and social sites so that students no longer have to use several applications to know what they need to accomplish that day. This application allows a user upon signing in to see a to-do list screen which will summarize the day's events and tasks. The user then can go to a calendar view of the day with built-in suggestions during free time based on user data and interests. The user can input events and tasks as well as link their outside calendar accounts in settings. The settings and profile pages allow for additional user options and the discover page gives access to AI suggested individuals.

**ERD:**




**Tech Stack**: Our team chose a HTML/JavaScript frontend with Supabase as a backend and a service.

**Frontend — HTML, CSS, and JavaScript:** HTML provides the structure of our screens, CSS controls the layout and appearance, and JavaScript handles interactions such as switching calendar views, adding events, and completing tasks.

**Backend and Database — Supabase with PostgreSQL:** Supabase hosts our relational database, which stores profiles, settings, interests, calendars, events, tasks, and completion status. Primary and foreign keys connect the tables based on our ERD.

This approach fits our team because it builds on the HTML, CSS, JavaScript, and SQL skills we are learning. Supabase reduces the work required to build and maintain a separate backend server, allowing us to focus on the calendar's features and user experience while still gaining experience with database design and application development.

## How to Get it Running

### Step 1: Download the project from GitHub

1. Go to the GitHub repository your instructor shared.
2. Click the green **Code** button near the top right.
3. Click **Download ZIP**.
4. Unzip the folder somewhere you can find it (your Desktop works fine).

**If you know how to use git:** git clone <repo-url> works too.

### Step 2: Open the app in your browser

Do not double-click the file to open it — that loads it as a file:// URL, which Chrome blocks from making requests to external services like Supabase. Use **Live Server** instead:

1. Open VS Code.
2. Install the **Live Server** extension if you do not have it yet: click the Extensions icon on the left sidebar (or press Ctrl+Shift+X / Cmd+Shift+X), search for "Live Server" by Ritwick Dey, and click Install.
3. Open the IS401_CalendarAppProject folder in VS Code (File → Open Folder).
4. Right-click calendar.html in the Explorer panel on the left → **Open with Live Server**.
5. Your browser should open automatically onto a sign in page.

Part 2 — Connect a real database with Supabase

Step 3: Create a Supabase account

1. Go to supabase.com and click **Start your project**.
2. Sign up with your GitHub account (easiest) or create an account with your email.
3. Verify your email if prompted.

Step 4: Create a new project

1. Once logged in, click **New project**.
2. Give your project a name (e.g. calendar-demo).
3. Set a database password — write it down somewhere, you may need it later.
4. Choose any region.

5. Click **Create new project** and wait about 30 seconds for it to set up.

Step 5: Create the user table

This is where you define what data your app will store.

1. In the left sidebar, click **Table Editor**.
2. Click **Create a new table**.
3. Name the table: user (lowercase, no spaces).
4. **Uncheck** "Enable Row Level Security (RLS)" — leave it off for now.
5. Supabase automatically adds id and created_at columns. Keep those.
6. Add these nine columns by clicking **Add column** for each:

| Name | Type | Default value | Nullable |
|---|---|---|---|
| user_id | uuid | *(leave blank)* | Set as primary key |
| name | text | *(leave blank)* | No |
| password | text | *(leave blank)* | No |

7. Click **Save**

Your database is ready.

Step 6: Get your credentials

These are the two pieces of information your app needs to talk to your database.

1. In the left sidebar, click **Project Settings** (the gear icon near the bottom).
2. You need two things from two different sections:
   **Your Project URL** — click **Data API** in the settings menu. Your URL is at the top of that page. It looks like https://abcdefgh.supabase.co.
   **Your publishable key** — click **API Keys** in the settings menu. Under '**Publishable key**,' copy the key that starts with sb_publishable_.... (Do not use the Secret key — that one has admin access and should never go in a frontend file.)

Keep this browser tab open — you will need both values in the next step.

Step 7: Add your credentials to the app

1. Open index.html in VS Code (right-click the file → Open with → VS Code, or open VS Code and drag the file in).
2. Use Find (Ctrl+F / Cmd+F) and search for YOUR_PROJECT_URL to jump straight to the right spot. You are looking for these three lines:

```
const SUPABASE_URL = 'YOUR_PROJECT_URL'   // <-- paste here
const SUPABASE_KEY = 'YOUR_ANON_KEY'      // <-- paste here
```

3. Replace 'YOUR_PROJECT_URL' with your Project URL (keep the quotes).

Warning

The URL must end in .supabase.co with **nothing after it**. If you copied it from the API settings page it may include /rest/v1/ at the end — delete that part or the app will not be able to reach your database.

4. Replace 'YOUR_ANON_KEY' with your publishable key (keep the quotes).
5. Save the file (Ctrl+S / Cmd+S).

Example of what it should look like after your edits:
const SUPABASE_URL = 'https://abcdefgh.supabase.co'
const SUPABASE_KEY = 'sb_publishable_xxxxxxxxxxxxxxxx...'

Step 8: Run SQL in Supabase

1. Copy the setup.sql file from GitHub.
2. Open SQL editor in Supabase, and paste the sql query and run it.
3. It should display “Success. No rows returned”.
4. Your Supabase should now be set up.

Step 9: Test it

1. Save index.html in VS Code (Ctrl+S / Cmd+S). Live Server will refresh the page automatically.
2. You should now be able to create an account by adding a username and password.
3. Refresh the page.
4. You should be able to log into your account now and your account credentials will appear in Supabase.